

CSC301-A

Module 1

Implementing a Basic Application Framework

Copyright ©
Symbolicon Systems
2008-2026

1 Encapsulation

A key feature of object-oriented programming is managing complexity. Building large and complex systems can be daunting as humans have limited capacity when dealing with size and complexity. Complexity can be managed by dividing a complex system into small and simple components that can then be individually designed, constructed and tested. Even though each component may still be moderately complex, it is much more manageable by encapsulating it within a class while exposing a simpler interface to access and integrate with the other components. Encapsulation will also protect the internals from unauthorized access and ensure data integrity within the component. When all the components are completed, it would then be very easy to construct the system from a reliable set of simple to use components. In this chapter, you will learn how to encapsulate the complexity of a common and useful component in a class and provide a very simple interface to access its functionality.

1.1 Application Logging

Many modern applications record information during execution called as logging and usually used by developers and administrators to debug or monitor the working state of the applications. In this section you will create a class to implement the application logging component. Since logging is a common component that can be used across applications, the class should be implemented in a separate class library.

Library project information

```
Project Name: Symbion
Project Type: Visual C# | Windows | Class Library
Location    : C:\CSDEV\SRC\
Solution    : Module1
```

Before you add a class to implementing logging, add the following enumeration that is passed to the logging class to determine the kind of message to write to the log. Then add the class that display the log message and other details to *Debug* window during debugging. Note that **Assert** and **WriteLine** methods are conditional and will only be called when a *debug version* of the library is used. Methods in **Trace** class works for both the debug and *release version*.

LogType enumeration: Symbion\LogType.cs

```
namespace Symbion {
    public enum LogType {
        Information,
        Warning,
        Error
    }
}
```

Logging component class: Symbion\DebugLogger.cs

```
using System;
using System.Diagnostics;
using System.IO;
using System.Reflection;

namespace Symbion {
    public class DebugLogger {
        private string _source;
        public string Source {
            get { return _source; }
            set {   Debug.Assert(value != null,
                    "Source property cannot be null.");
                _source = value;}
            }
        }
        public DebugLogger() {
            Assembly assembly = Assembly.GetEntryAssembly();
            Source = Path.GetFileNameWithoutExtension(assembly.Location);
        }
        public void Write(string message, LogType logType) {
            Debug.WriteLine($"[{Source}] {logType}(\"{message}\")");
        }
    }
}
```

Source property is commonly the name of the application using the class and should not be null. Passing null to it is considered a bug and will be detected by the **Assert** method during debugging. The constructor ensures that the source is never null even when the object is created by automatically retrieving the application name. Use an **Assembly.GetEntryAssembly** method to access application assembly and the name extracted from **Location** property that returns the full path of the assembly. Since we only need the name, you can call **GetFileNameWithoutExtension** method from the **Path** class to extract that part from the entire path. Add an application assembly project to the solution to test out the logging class using the following information.

Application project information

```
Project Name: Encapsulation1
Project Type: Visual C# | Windows | Console Application
Location    : C:\CSDEV\SRC\Module1
```

Main program to test the logging class: Encapsulation/Program.cs

```
using System;

class Program {
       static void Main() {
         DebugLogger logger = new Symbion.DebugLogger();
         Console.WriteLine(logger.Source);
         logger.Write("This is the 1st message.", LogType.Information);
         logger.Write("This is the 2nd message.", LogType.Warning);
         logger.Write("This is the 3rd message.", LogType.Error);
    }
}
```

Run the application in debug mode and you should see the messages appearing in the *Debug output window*. Once you have tested it, let us further simplify usage of the class by setting default values and add helper methods.

1.2 Default Values & Helper Methods

The common log message type is **LogType.Information** so we will assign this as the default value for **logType** parameter in the **Write** method. Thus there is no more need to pass in the second parameter if you intend to write this type of message and the compiler generates code to pass the second parameter.

Setting the default log type: Symbion\DebugLogger.cs

```
public void Write(string message,
    LogType logType = LogType.Information) {
    Debug.WriteLine(string.Format(LogFormat,
        Source, logType, message));
}
```

Using the default value: Encapsulation1\Program.cs

```
logger.Write("This is the 4th message.");
```

Alternatively you can add additional methods that help you to pass the log type so that the compiler would not have to generate the code each time the **Write** method is called.

Adding helper methods: Symbion\DebugLogger.cs

```
public void Message(string message) { Write(message, LogType.Information); }
public void Warning(string message) { Write(message, LogType.Warning); }
public void Failure(string message) { Write(message, LogType.Error); }
```

Using helper methods: Encapsulation1\Program.cs

```
logger.Message("Operation completed successfully.");
logger.Warning("Operation may not have succeeded.");
logger.Failure("Operation has failed.");
```

Encapsulation also allows you to update, change or expand the contents of the class without affecting the code that uses it as long as the class interface it uses does not change. The code does not even need to be re-compiled as the class is distributed in a separate assembly. However, since the code is directly using the class, you cannot replace the class with another without changing the code and re-compiling it. In order for the same code to be able to use different classes at run-time you will first need to abstract the class away from the code. You can do this by separating the interface and implementation through abstraction.

2 Abstraction & Inheritance

Abstraction is where the design of a component is separated from its implementation. There are many advantages for abstraction. Assembly dependency is unidirectional so classes that need bidirectional access to each other have to be compiled within the same assembly. However the design can compiled in one assembly and the code that implements the design or uses the design can be compiled in same or other assembly. One design can have multiple implementations supporting exchangeable components. It allows the possibility of polymorphism where one component can be substituted by another compatible component at run-time to build a dynamic system. Abstraction and polymorphism can be implemented using interfaces or base classes but interface is always used to provide the highest level of abstraction. A class is limited to only one base class but can implement any number of interfaces.

2.1 Interface & Abstraction

You can now add an interface to **Symbion** project named as **Ilogger**. Declare all the property and methods from **DebugLogger** in the interface as shown below. You can mark **DebugLogger** as implementing the interface and you can then update the code to use the object through the interface.

Interface declaration: Symbion\ILogger.cs

```
using System;

namespace Symbion {
    public interface ILogger {
        string Source { get; set; }
        void Write(string message, LogType logType = LogType.Information);
        void Warning(string message);
        void Message(string message);
        void Failure(string message);
    }
}
```

Class implementing the interface: DebugLogger.cs

```
public class DebugLogger : ILogger {
```

Using object through interface: Abstraction1\Program.cs

```
ILogger logger = new DebugLogger();
```

The key purpose of abstraction is so that you can implement multiple classes having the same interface. You can write code that can work with objects from any class through the interface. The classes does not require any relationship with each other except that they implement the same interface. Only three types of members can be declared in an interface; *event*, *property* and *method*.

2.2 Base Class & Inheritance

Sometimes different classes may still contain the same or similar content. It becomes cumbersome to have to write, maintain and update the same content across multiple classes. To remove redundancy, you can create a separate class to place the content instead. Making this class as the base class will allow the content to be inherited into one or more derived classes. Interface is purely abstract so you cannot anything from an interface except for the declaration.

Base class used for inheritance: Symbion\BaseLogger.cs

```
using System;
using System.Diagnostics;
using System.Reflection;
using System.IO;

namespace Symbion {
    public abstract class BaseLogger : Ilogger {
        private string _source;
        public virtual string Source {
            get { return _source; }
            set {   Debug.Assert(value != null,
                    "Source property cannot be null.");
                _source = value; }
        }
        public BaseLogger() {
            Assembly assembly = Assembly.GetEntryAssembly();
            Source = Path.GetFileNameWithoutExtension(assembly.Location);
        }
        public abstract void Write(string message,
            LogType logType = LogType.Information);

        public void Message(string message) {
            Write(message, LogType.Information);
        }
        public void Warning(string message) {
            Write(message, LogType.Warning);
        }
        public void Failure(string message) {
                    Write(message, LogType.Error);
        }
    }
}
```

Since you may implement different logging components, you can add a base class for them since most of the code that has been implemented in **DebugLogger** is usable in other loggers as well. Rename DebugLogger to **BaseLogger** to turn into a base class. You can mark private members as **protected** to allow derived classes to access them directly if required. Use **virtual** to allow derived classes to override members and for members that cannot be inherited can be marked **abstract**. If the class have at least one abstract member the class must be marked abstract as well. You can now create a new DebugLogger class that only require to override the **Write** method. The other members can be inherited from the base class.

Extending from base abstract class: DebugLogger.cs

```
using System;
using System.Diagnostics;

namespace Symbion {
    public class DebugLogger : BaseLogger {
        public override void Write(string message,
            LogType logType = LogType.Information) {
            Debug.WriteLine("[{Source}] {logType}(\"{message}\")");
        }
    }
}
```

When you extend a class, you always get full inheritance. That means **DebugLogger** will be fully compatible with **BaseLogger** so it is still possible to use a base class for abstraction as shown below. Since **BaseLogger** also has all the members required for **ILogger** you can also set that it implements the interface. This means that all classes that extend **BaseLogger** will automatically support **ILogger** as well.

Using base class for abstraction: Abstraction1\Program.cs

```
BaseLogger logger = new DebugLogger();
```

3 Activation & Polymorphism

Abstraction only allows you to access an existing object. The object has to be created first from the class. However the moment you hard-code the name of the class in the source code, the object would always be an instance of that class. You need to modify the code to change the class name and rebuilt the assembly to create an object from a different class. Polymorphism is where you can substitute an object with a different type of object without having to change and recompile code. To accomplish this in our code, you need to use dynamic object activation to instantiate the object and not use the **new** keyword.

3.1 Object Activation

The easiest way to dynamically create an object is to use the **Activator** class. It has many ways to create an object that do not require hard-coded class names. You can create an object by providing assembly name and type name to the **CreateInstance** method. If the assembly is not loaded, it will be loaded automatically. However the method does not return the real object but an **ObjectHandle** instead. You still need to call an **Unwrap** method to access the object and the reason for doing so will be explained in a later module. Since the names are just text strings, they can be stored externally rather than using them literally in the source code. This will allow you to create a different type of object or load in a different assembly without rebuilding the code.

Dynamic object activation: Activation1\Program.cs

```
ILogger logger = (ILogger)Activator.CreateInstance(
       "Symbion","Symbion.DebugLogger").Unwrap();
```

However do not expect developers to write so much code to instantiate an object thus you need to simplify the usage of the **Activator** by implementing another class. Using one class or object to create objects from another class is nothing new in software development as there is a design pattern for it called the *factory method*.

3.2 Factory Method

You will now implement a factory class to encapsulate the complexity that is required and return an instance of a logging class. The actual assembly name and class name is stored in the application configuration file. However, it should not be compulsory that all applications must provide this information. If it is not provided, the factory would use **DebugLogger** as the default.

To test this add an application configuration file into the application assembly project. Then add a setting where the value contains the type and assembly name and the key to fetch it. The key can be anything and value can be in any format you desire. For our example, we use the full interface name as the key and the value contains the names separated by a semicolon. Even though the file is named **App.config**, when you compile the assembly, it will have the same name as the assembly but extended with the **.config** extension which in this case is **Abstraction1.exe.config**.

Application configuration file: Abstraction1\App.config

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
    <appSettings>
        <add key="Symbion.ILogger" value="Symbion.DebugLogger; Symbion"/>
    </appSettings>
</configuration>
```

Even though only applications have configuration files, it can also be accessed from all assemblies with **ConfigurationManager**. Reference the **System.Configuration** assembly to use it. You can implement a **static** class so that you don't need to create an object from the factory in order to get a logging object.

Factory method instantiate loggers: Symbion\LoggerFactory.cs

```
using System;
using System.Configuration;

namespace Symbion {
    public static class LoggerFactory {
        public const string DefaultLogger = "Symbion.DebugLogger; Symbion";
        private static readonly string _typeName;
        private static readonly string _assemblyName;
        static LoggerFactory() {
            string key = typeof(ILogger).FullName;
            string value = ConfigurationManager.AppSettings[key];
            if (value == null) value = DefaultLogger;
            string[] fields = value.Split(';');
            _typeName = fields[0].Trim();
            _assemblyName = fields[1].Trim();
        }
        public static ILogger CreateInstance() {
            return (ILogger)Activator.CreateInstance(
                _assemblyName, _typeName).Unwrap();
        }
    }
}
```

Instancing logging object through factory: Activation1\Program.cs

```
ILogger logger = LoggerFactory.CreateInstance();
```

3.3 Polymorphism

To demonstrate polymorphism, you will now need to create additional logging classes. However you can implement them in another assembly. To show that we can easily access new loggers without having to compile the application or the **Symbion** library. All that is needed is to update the configuration file. Create a new class library using the following information, enter the assembly version details and then signed with the same key file from **Symbion** project.

Library project information

```
Project Name: Symbion.Loggers
Project Type: Visual C# | Windows | Class Library
Location    : C:\CSDEV\SRC\Module1
```

Another implementation of ILogger: FileLogger.cs

```
using System;
using System.IO;
namespace Symbion.Loggers {
    public class FileLogger : BaseLogger {
        private string _filename;
        public override string Source {
            get { return base.Source; }
            set {   base.Source = value;
                _filename = Path.ChangeExtension(value, ".log");
            }
        }
        public override void Write(string message, LogType logType) {
            string text = $"\"{DateTime.UtcNow}\",\"{logType}\",\"{message}\"\r\n";
            File.AppendAllText(_filename, text);
        }
    }
}
```

The previous FileLogger class write the log message into a file. The next class named as named **EventLogger** will log messages to the system event log instead. Once this is done, you can add the reference to the **Symbion.Loggers** library in the application so that it is deployed to the application folder.

Another implementation of ILogger: EventLogger.cs

```
using System;
using System.Diagnostics;
namespace Symbion.Loggers {
    public class EventLogger : BaseLogger {
        public override void Write(string message,
            LogType logType = LogType.Information) {
            EventLogEntryType entryType = EventLogEntryType.Information;
            if (logType == LogType.Error) entryType = EventLogEntryType.Error;
            else if (logType == LogType.Warning) entryType = EventLogEntryType.Warning;
            EventLog.WriteEntry(Source, message, entryType);
        }
    }
}
```

Referencing assemblies does not mean that they will automatically be loaded if you do not use the types in the assemblies. You can use the following code to check what assemblies are actually loaded at the start of the application.

Code to check assemblies currently loaded: Symbion\DebugHelper.cs

```
public static class DebugHelper {
    public static void ShowLoadedAssemblies() {
        AppDomain domain = AppDomain.CurrentDomain;
        Assembly[] assemblies = domain.GetAssemblies();
        foreach (Assembly assembly in assemblies)
            Console.WriteLine(assembly.FullName);
    }
}
```

You can now build the project and then modify the application configuration file in the application output folder to use a different logging class. There is no need to re-build the solution each time the user wants to change the logger.

4 Service-Oriented Architecture

A large software system is usually constructed by an entire development team rather than a single programmer. The software system make be designed as an integrated set of modules where each module can be constructed by a different team member. It is possible that modules each work independently or they may also have to integrate to one another. It is important that each module will be developed as a separate assembly so that you do not need to recompile the entire software system when there are small changes to certain modules. It is more efficient to compile only changed modules. Even if this is the best option you will encounter dependency issues when modules need to communicate with each other. This is because assembly dependency is unidirectional. This means that if assembly A references assembly B, then B will not be able to reference assembly A. This is because you cannot compile A if B has not been compiled first and you cannot compile B as A is not compiled as well. Abstraction can resolve bidirectional relationships but objects have to be created somewhere and the objects have to locate each other during runtime before they can communicate with each other through interfaces. Dependency injection is a pattern to allow objects to resolve references to each other so they can communicate at runtime even without knowing which types and where and when objects are created. One way of implement dependency injection is to use an IOC container.

4.1 ServiceRepository

It is not compulsory for containers to store a specific kind of object, but this example you can apply a service-oriented architecture for the container. In this architecture, each module may provide a set of services. When a module is initialized, it can add its services into the container. A class that requires using those services may lookup the services from the container. Automation can also be applied where the container will update the class automatically when any of the services required are placed into the container. You can add an interface to distinguish service classes from normal classes. You may consider that logging is a service that can be shared across modules.

Declaring interface for services: Symbion\IService.cs

```
namespace Symbion {
    public interface IService {
    }
}
```

Logging is now a specialized kind of service: ILogger.cs

```
namespace Symbion {
    public interface ILogger : IService {
            :
```

All services may need to share some common features or code so it make sense to create a base class to provide partial or full implementation for them so that they can be inherited instead of having to re-implement the same features in all services.

Generic base class for all services: Symbion\BaseService.cs

```
using System;

namespace Symbion {
    public abstract class BaseService : IService {

    }
}
```

Extending from a generic base class: BaseLogger.cs

```
public abstract class BaseLogger : BaseService, ILogger {
            :
```

You can now implement a class that keeps a repository of service objects. An object can be injected into the container based on what kind of services that it provides. Anywhere the service will be required you can fetch the object based on the type of service. It does not matter what is the class and where is implemented. Add a class named **ServiceRepository** to Symbion.

A simple service container: Symbion\ServiceRepository.cs

```
using System;
using System.Collections.Generic;

namespace Symbion {
    public static class ServiceRepository {
        private static Dictionary<Type, IService> _services =
            new Dictionary<Type, IService>();
        public static bool Add(Type serviceType,IService serviceObject) {
            if (!_services.ContainsKey(serviceType)) {
                _services.Add(serviceType, serviceObject); return true; }
            return false;
        }
        public static IService Get(Type serviceType) {
            IService serviceObject = null;
            _services.TryGetValue(serviceType, out serviceObject);
            return serviceObject;
        }
    }
}
```

The class contain a dictionary for registering services based on their service type. The **Add** method will be used to register the service while the **Get** method is used to locate the correct service based on the service type. The class is static so that you do not need to instantiate the container before using it. The following shows the code to place the logger into the container and to fetch the logger from the container. The wonderful thing about this is that the code can be compiled into different assemblies that do not have to reference each other. They only have to reference an assembly where the interface is declared which can be constructed just for interfacing.

Example code to add a logger into the container: Dependency1\Program.cs

```
ServiceRepository.Add(typeof(ILogger), LoggerFactory.CreateInstance());
```

Example code to fetch logger from the container

```
ILogger logger = (ILogger)ServiceRepository.Get(typeof(ILogger));
if(logger != null) logger.Write("Logger service located!");
```

While previous code works, the use of **typeof** operator and also typecasting required when fetching the object makes the code looked complicated. You can implement the methods as generic so that you can pass the exact type required to the methods. The methods can adapt to the types required and you can embed **typeof** and typecasting operations within the methods.

Generic version of container methods: Symbion\ServiceRepository.cs

```
public static bool Add<TService>(TService serviceObject)
    where TService : IService {
    Type serviceType = typeof(TService);
    if (!_services.ContainsKey(serviceType)) {
        _services.Add(serviceType, serviceObject);
        return true; } return false;
}

public static TService Get<TService>() where TService : IService {
    IService serviceObject = null;
    Type serviceType = typeof(TService);
    _services.TryGetValue(serviceType, out serviceObject);
    return (TService)serviceObject;
}
```

Simpler code to add a logger into the container

```
// ServiceRepository.Add<ILogger>(new DebugLogger());
ServiceRepository.Add(LoggerFactory.CreateInstance());
```

Example code to fetch logger from the container

```
ILogger logger = ServiceRepository.Get<ILogger>();
if(logger != null) logger.Write("Logger service located!");
```

The code to use the container is much simpler due to generic methods. Since the Add method is static and ServiceRepository is a static class, you can turn the method into an extension method of **IService**. This means that the method will automatically appear on any object that is a service.

Turning a static method into an extension method

```
public static bool Add<TService>(this IService serviceObject) {
        :
```

Simplest code to add where container class name is not required

```
// new DebugLogger().Add<ILogger>();
LoggerFactory.CreateInstance().Add();
```

4.2 Authorization

Large scale applications are not built for one user. An enterprise application is built for everyone in an organization. Different users perform different tasks and should have different levels of security clearances so they cannot get access to something that they are not supposed to have. Thus authorization will be a common service. Since this is common feature for applications, they will be implemented in **Symbion**. Add an interface called **IAuthorization** containing the following members.

Role-based security service: Symbion\IAuthorization.cs

```
using System;
using System.Collections.Generic;

namespace Symbion {
    public interface IAuthorization : IService {
        string UserName { get; }
        bool IsInRole(string roleName);
        bool IsInAnyRoles(IEnumerable<string> roleNames);
        bool IsInAllRoles(IEnumerable<string> roleNames);
    }
}
```

.NET applications uses principal system for authorization. However just as we can freely choose a different authorization service, it can also choose the principal system to use. When building Windows applications, you can then choose to use Windows principal. Set the principal policy for current application to **WindowsPrincipal** and it can then be available to every thread through the **CurrentPrincipal** property.

Authorization with principal object: PrincipalAuthorization.cs

```
using System;
using System.Collections.Generic;
using System.Security.Principal;
using System.Threading;
namespace Symbion {
    public class PrincipalAuthorization : BaseService, IAuthorization {
        private IPrincipal _principal;
        public string UserName {
            get { return _principal.Identity.Name; }
        }
        public PrincipalAuthorization() {
            _principal = Thread.CurrentPrincipal;
        }
        public bool IsInRole(string roleName) { return _principal.IsInRole(roleName); }
        public bool IsInAnyRoles(IEnumerable<string> roleNames) {
            foreach (string roleName in roleNames)
                if (_principal.IsInRole(roleName)) return true;
            return false;
        }
        public bool IsInAllRoles(IEnumerable<string> roleNames) {
            foreach (string roleName in roleNames)
                if (!_principal.IsInRole(roleName))  return false;
            return true;
        }
    }
}
```

Adding authorization service into the container: Dependency1\Program.cs

```
AppDomain.CurrentDomain.SetPrincipalPolicy(PrincipalPolicy.WindowsPrincipal);
new PrincipalAuthorization().Add<IAuthorization>();
```

Fetching and using authorization service

```
var auth = ServiceRepository.Get<IAuthorization>();
Console.WriteLine(auth.UserName);
Console.Writeline(auth.IsInRole("Administrators"));
Console.Writeline(auth.IsInRole("Banking"));
```

For security concerns, you should also try to use domain roles instead of local roles as a user might be able to obtain administrative privileges to the local computer to add themselves to the required roles but not when on a remote domain server.

5 Modules & Serialization

5.1 Modules

You can think a module as a dynamic component of an application loaded only when necessary. The module can contain a set of services that can be either used internally or exposed externally for integration to other modules. A module may contain views to allow user to interact visually with the module and its services. To allow modules to perform initialization and finalization operations, the interface should expose an **Init** and **Exit** method. You can implement a base class for this interface to provide blank implementations since not every module require implementing the members.

Basic interface for modules: Symbion\IModule.cs

```
namespace Symbion {
    public interface IModule {
        void Init();
        void Exit();
    }
}
```

Base class for modules: Symbion\BaseModule.cs

```
using System;

namespace Symbion {
    public class BaseModule : IModule {
        public virtual void Init() { }
        public virtual void Exit() { }
    }
}
```

5.2 Serialization

Smart client applications commonly check what modules are available and determine which modules is to be loaded depending on the user is running the application. It would not be appropriate to load modules that the user will never use or safe to allow the user to access modules that they are not authorized to use. Each module should have a class used to initialize a module when loaded. Add a **BankingModule** class to **SymBank.Banking** project. Extend **BaseModule** to inherit the default code. We will add more code to this class later.

Basic implementation of a module class: BankingModule.cs

```
using System;
using Symbion;

namespace SymBank.Banking {
    public class BankingModule : BaseModule {
    }
}
```

We can implement a loader to dynamically load modules. However, before this can be done, we need a data model to store information about what modules are available, who are authorized to use them and where are the location of the modules and also the name of the module class. While you can use application configuration file to store simple settings but you have to perform a lot more work to use it for storing custom objects. It will be simpler to implement your data model in code and use serialization to store and load objects. In this example, we will use XML serialization.

You can add a **ModuleItem** class to the Symbion library. The purpose of this class is to store the location and name of the module, and also a list of roles to determine the users using this module. If the user is not part of any of the roles, the module will not be loaded. This is where we can make use of the authorization service that we have implemented previously. You can add a **ModuleList** class to the library. This class is used to store a collection of **ModuleItem** elements. Add a static **Load** method to de-serialize an instance from an XML document by using the **XmlSerializer**. Also provide a method to serialize an instance into an XML document so that it will be possible to create the document by writing code. The XML document can then be edited outside of the application.

Class to store information about a single module: ModuleItem.cs

```
using System;
using System.Collections.Generic;
namespace Symbion {
    public class ModuleItem {
        public string Name { get; set; }
        public string Path { get; set; }
        public List<string> Roles { get; set; }
        public ModuleItem() { Roles = new List<string>(); }
    }
}
```

Class to store a list of module items: ModuleList.cs

```
using System;
using System.Collections.Generic;
namespace Symbion {
    public class ModuleList {
        public List<ModuleItem> Items { get; set; }
        public ModuleList() { Items = new List<ModuleItem>(); }
    }
}
```

Methods to serialize and de-serialize a ModuleList from XML

```
public void Save(string path) {
    XmlSerializer serializer = new XmlSerializer(typeof(ModuleList));
    FileStream stream = new FileStream(
             path, FileMode.Create, FileAccess.Write);
    try { serializer.Serialize(stream, this); }
      finally { stream.Close(); }
}

public static ModuleList Load(string path) {
    if (!File.Exists(path)) return new ModuleList();
    XmlSerializer serializer = new XmlSerializer(typeof(ModuleList));
    FileStream stream = new FileStream(path, FileMode.Open, FileAccess.Read);
    try { return (ModuleList)serializer.Deserialize(stream); }
    finally { stream.Close(); }
}
```

5.3 Serialization Format

If you want to customize the serialization then you need to serialize an example first so that you can see the serialized XML format. Add a console application to the same solution named **Serialize1** and write code to construct the model and call **Save**.

Serializing our object model to XML: Serialization1\Program.cs

```
using System;
using Symbion;
class Program {
    static void Main() {
        ModuleList list = new ModuleList();
        ModuleItem item = new ModuleItem();
        item.Name = "SymBank.Banking.BankingModule";
        item.Path = "SymBank.Banking.dll";
        item.Roles.Add("Administrators");
        item.Roles.Add("Banking");
        list.Items.Add(item);
        list.Save("Modules.xml");
    }
}
```

Output format of the above serialization: Modules.xml

```
<?xml version="1.0"?>
<ModuleList xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <Items>
    <ModuleItem>
      <Name>SymBank.Banking.BankingModule</Name>
      <Path>SymBank.Banking.dll</Path>
      <Roles>
        <string>Administrators</string>
        <string>Banking</string>
      </Roles>
    </ModuleItem>
  </Items>
</ModuleList>
```

The above format is usable but we are going to change the format slightly so it can be edited easier by hand. First change the name of the root element from *ModuleList* to *ModuleCatalog* by attaching an **XmlType** attribute to the class. Also change the name of the *Items* property to *Modules*. Use **XmlElement** and **XmlAttribute** to change the name of properties but since this is a collection property, use the **XmlArray** attribute instead to change the name.

We also prefer that a *ModuleItem* is serialized as *Module*. It would be simpler to edit if we made *Name* and *Path* as XML attributes rather than as elements. This can be done using **XmlAttribute**. Change also the name of each item in the *Roles* collection which is now serialized as *string*. You can change name of serialized items in an array or collection using **XmlArrayItem**. Anything that you do not want to serialized, you can mark it using **XmlIgnore**. By default all public fields and properties will be serialized when using XML serialization.

Changing serialized type and property name: ModuleList.cs

```
[XmlType("ModuleCatalog")]
public class ModuleList {
    [XmlArray("Modules")]public List<ModuleItem> Items { … }
                :
```

Customizing ModuleItem serialization: ModuleItem.cs

```
[XmlType("Module")]
public class ModuleItem {
    [XmlAttribute]public string Name { … }
    [XmlAttribute]public string Path { … }
    [XmlArrayItem("Role")]public List<string> Roles { … }
                :
```

Output of new serialization format

```
<?xml version="1.0"?>
<ModuleCatalog
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <Modules>
        <Module
            Name="SymBank.Banking.BankingModule"
            Path="SymBank.Banking.dll">
            <Roles>
                <Role>Administrators</Role>
                <Role>Banking</Role>
            </Roles>
        </Module>
    </Modules>
</ModuleCatalog>
```

You can now implement a **ModuleLoader** class in Symbion project. The class keeps a collection of **IModule** objects that have been instantiated from the loaded modules. The **Init** method will be used to initialize all modules and the **Exit** method to finalize all modules. The **Load** method is called with the path to the XML file containing the **ModuleList** which will then load in all authorized modules.

ModuleLoader class implementation: Symbion\ModuleLoader.cs

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Reflection;
using System.IO;

namespace Symbion {
    public static class ModuleLoader {
        private static List<IModule> _modules = new List<IModule>();
        public static void Init() {
            foreach (IModule module in _modules) module.Init(); }
        public static void Exit() {
            foreach (IModule module in _modules) module.Exit(); }
        public static void Load(string path) { }
    }
}
```

You can use **Assembly.LoadFrom** method to load in an assembly dynamically from a path. Once an assembly is loaded you can locate a **Type** by name using the **GetType** method or **GetExportedTypes** method to obtain all public types in the assembly.

Implementation of the Load method

```
IAuthorization auth = ServiceRepository.Get<IAuthorization>();
ILogger logger = ServiceRepository.Get<ILogger>();
ModuleList list = ModuleList.Load(path);
foreach (ModuleItem item in list.Items) {
    if (item.Roles.Count > 0 && !auth.IsInAnyRoles(item.Roles)) {
        Debug.WriteLine(string.Format("User not authorized for module {0}.", item.Path));
        continue;
    }
    if (!File.Exists(item.Path)) {
        logger.Failure(string.Format("Cannot locate module {0}.", item.Path));
        continue;
    }
    Assembly assembly = null;
    try { assembly = Assembly.LoadFrom(item.Path); }
    catch (Exception ex) {
        logger.Failure(string.Format(
            "Error '{0}' occurred loading module {1}.", ex.Message, item.Path));
        continue;
    }
    Type moduleType = assembly.GetType(item.Name);
    if (moduleType == null) {
        logger.Failure(string.Format(
            "Cannot find class {0} in module {1}.", item.Name, item.Path));
        continue;
    }
    try {
        _modules.Add((IModule)Activator.CreateInstance(moduleType));
        Debug.WriteLine(string.Format("Module {0} loaded successfully.", item.Path));
    }
    catch (Exception ex) {
        logger.Failure(string.Format(
            "Error '{0}' instancing {1} in module {2}.", ex.Message, item.Name, item.Path));
    }
}
```

You can now make use of the **ModuleLoader** to load in any dynamic modules for an application. All the modules can be initialized and finalized by using the **Init** and **Exit** methods. You can add a new library project to the solution to implement an example module. Since modules may contain views you need to create a library project that is setup to have UI components. Windows support two UI systems that you can use which is *Windows Form* and *Windows Presentation Foundation*. You will be using WPF as our UI so create a *WPF User Control Library* project.

Library project information

```
Project Name: SymBank.Banking
Project Type: Visual C# | Windows | WPF User Control Library
Location    : C:\CSDEV\SRC\Module1
```

Service Interface: SymBank.Banking\Services\IAccountController.cs

```
public interface IAccountController : IService  { }
```

Service Interface: SymBank.Banking\Services\ITransactionController.cs

```
public interface ITransactionController : IService { }
```

Service class implementation: SymBank.Banking\Controllers\BankingController.cs

```
public class BankingController : IAccountController, ITransactionController { }
```

Example module initializing services: SymBank.Banking\BankingModule.cs

```
namespace SymBank.Banking {
    public class BankingModule : BaseModule {
            protected override void Init() {
                base.Init();
                var obj = new BankingController();
                obj.Add<IAccountController>();
                obj.Add<ITransactionController>();
            }
    }
}
```

Loading and finalizing modules: Modules1\Program.cs

```
AppDomain.CurrentDomain.SetPrincipalPolicy(PrincipalPolicy.WindowsPrincipal);
new PrincipalAuthorization().Add<IAuthorization>();
LoggerFactory.CreateInstance().Add();
ModuleLoader.Load("Modules.xml");
ModuleLoader.Init();
ModuleLoader.Exit();
```

Add *Modules.xml* file to the application project. Use properties window to ensure that file is set as **Content** and *Copy To Output Directory* option is set to **Copy if newer**. This ensures the file will be deployed together with the application. As **DebugLogger** is used by default, check the debug output window to see the results. If the user is authorized, the module will be loaded and initialized.

6 Attributes & Automation

6.1 .NET Attributes

You can use .NET attributes to attach information to program element that includes the assembly, class, structure, interface, delegate, field, event, property and method. Each .NET attribute is an object that is instantiated from a class that is extended from a base **Attribute** class. This will allow anyone to create their own custom attributes. Reflection can be used during runtime to obtain custom attributes from types. When implementing your custom attribute class, the class name must always end with **Attribute**. To limit which program element that your attribute can be attached to and also if the attribute can be attached more than once to a single element, use the **AttributeUsage** attribute on your class. You can decide whether an attribute can only be used once on each element by setting **AllowMultiple** to false. The following is an attribute class named **ServiceAttribute** that can be attached to a class. It is up to you to decide if you need to store data in your attributes. If so you can then create a custom constructor to allow the data to be passed in easily and provide properties to retrieve the data later. You may also allow properties to be set separately rather than passed through the constructor.

Service attribute class: Symbion\ServiceAttribute.cs

```
namespace Symbion {
    [AttributeUsage(AttributeTargets.Class, AllowMultiple = true)]
    public sealed class ServiceAttribute : Attribute {
        private Type _serviceType;
        public Type ServiceType {
            get { return _serviceType; }
            set { _serviceType = value; }
        }
        public ServiceAttribute() { }
        public ServiceAttribute(Type serviceType) {
            _serviceType = serviceType;
        }
    }
}
```

Assigning attribute to class: SymBank.Banking\Controllers\BankingController.cs

```
    [Service(typeof(IAccountController))]
    [Service(typeof(ITransactionController))]
//  [Service(ServiceType = typeof(IAccountController))]
//  [Service(ServiceType = typeof(ITransactionController))]
    public class BankingController : IAccountController, ITransactionController {
    }
```

6.2 Automation

Operations such as creating and injecting objects into a container can be automated through an application framework. Previously we implemented an **ServiceAttribute** that can be attached to any class. Our application framework can specifically look for this type of attribute by examining all types exported from an assembly. You can call **GetCustomAttributes** method on any **Type** to locate specific attribute types. The method returns an array since you may be able to attach multiple instances of an attribute if **AllowMultiple** is **true**. You can determine if inherited attributes is allowed using a second parameter. We will now add a new method to **ServiceRepository** class. This method check all the types in assembly containing the module class to locate all classes that have been attached the attribute. It will then create an instance of that object to be registered as a service by using the service type in the attribute. Change the **Init** method in ModuleLoader to inject every module loaded.

Auto-instancing and registration of services: ServiceRepository.cs

```
public static void AddServices(this IModule module) {
    Assembly assembly = module.GetType().Assembly;
    Type[] types = assembly.GetExportedTypes();
    foreach (Type type in types) {
        if (!type.IsClass) continue;
        var attributes = (ServiceAttribute[])
            type.GetCustomAttributes(typeof(ServiceAttribute), false);
        if (attributes.Length == 0) continue;
        var instance = (IService)Activator.CreateInstance(type);
        foreach (var attribute in attributes) {
            Type serviceType = attribute.ServiceType;
            if (!_services.ContainsKey(serviceType))
                _services.Add(serviceType, instance);
        }
    }
}
```

Auto-injection of module services: ModuleLoader.cs

```
public static void Init() {
    foreach (IModule module in _modules) {
        module.Init();
        module.AddServices();
    }
}
```

So now there is no need for a module to have to instantiate and register any services. It will be automatically be done just by attaching the attribute to any service class. With automation, developers can write less code to perform standard operations and having less code usually means less bugs.